

Outline

Loss vs. metric: what's the difference?

Loss. The function the model *optimizes during fitting*. Must be differentiable for gradient-based methods.

Examples: MSE for OLS, log loss for logistic regression.

Metric. The number you *report* to humans. Doesn't have to be differentiable.

Examples: RMSE in kAMD, MAE, MAPE, R^2 .

Loss and metric *can* be the same (often are for regression) - but they don't have to be. You might train to minimize MSE because of its math properties, then report MAE because stakeholders prefer “typical error in real units.”

The key question: *what kind of mistake hurts you most?* The right metric depends on the answer.

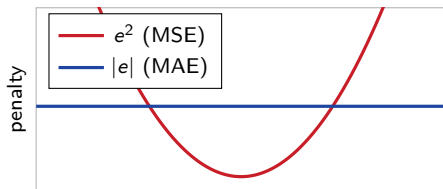
- ▶ One big error vs. many small ones?
- ▶ Errors on cheap items vs. expensive items?
- ▶ Over- vs. under-prediction?

MSE, RMSE, MAE: how the penalty shapes differ

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- **MSE/RMSE** penalize residuals *quadratically*: one error of 100 contributes 10 000; ten errors of 10 contribute 1000 total. Big errors dominate.
- **MAE** penalizes *linearly*: one big and ten small contribute the same total. Robust.
- Report RMSE to humans (units of y).
Use MSE internally (smoother for optimization).



residual $e = y - \hat{y}$
data + residuals



Same residuals; the penalty curve shape decides each metric.

MAE and MedAE: robust variants

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \text{MedAE} = \text{median}_i |y_i - \hat{y}_i|$$

- ▶ **MAE.** Same units as y . Penalizes errors *linearly* - an error of 100 contributes 100, an error of 10 contributes 10. Outliers don't dominate.
- ▶ **MedAE.** The median absolute residual. Even more robust than MAE - half the errors are at most this big.
- ▶ Where each shines: *MAE* for real-estate, finance, anything with outlier-heavy targets. *MedAE* for “typical case” reporting (“half of my predictions are within 30 kAMD”).

```
from sklearn.metrics import mean_absolute_error, median_absolute_error
mae      = mean_absolute_error(y, y_pred)
medae    = median_absolute_error(y, y_pred)
```

Predict-first: which metric notices the outlier?

10 predictions of Yerevan rent. 9 are off by 1 kAMD, 1 is off by 100 kAMD (a luxury apartment your model missed).

Predict-first: which metric notices the outlier?

10 predictions of Yerevan rent. 9 are off by 1 kAMD, 1 is off by 100 kAMD (a luxury apartment your model missed).

Predict the four metrics:

► MSE? RMSE? MAE? MedAE?

Predict-first: which metric notices the outlier?

10 predictions of Yerevan rent. 9 are off by 1 kAMD, 1 is off by 100 kAMD (a luxury apartment your model missed).

Predict the four metrics:

► MSE? RMSE? MAE? MedAE?

Metric	Value	Interpretation
MSE	$\frac{9 \cdot 1^2 + 1 \cdot 100^2}{10} = 1000.9$	dominated by the outlier
RMSE	$\sqrt{1000.9} \approx 31.6$	"typical error 32 kAMD" (misleading)
MAE	$\frac{9 \cdot 1 + 100}{10} = 10.9$	visible but moderate
MedAE	median = 1	outlier completely invisible

Lesson: same model, four different stories. The metric you pick *is* the conclusion you'll report.

RMSLE: for skewed positive targets

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2}$$

- ▶ Penalizes *relative* errors, not absolute. Predicting 100 when truth is 110 has the same RMSLE as predicting 1000 when truth is 1100 (both $\sim 10\%$ off).
- ▶ Penalizes under-prediction harder than over-prediction (the log function bends that way).
- ▶ Default for prices, sales, counts - anything strictly positive and skewed. Kaggle's go-to for housing-price and demand-forecasting competitions.
- ▶ Pairs with the `log1p` transform from `L01c`.

```
from sklearn.metrics import mean_squared_log_error
import numpy as np
rmsle = np.sqrt(mean_squared_log_error(y, y_pred))
```


Magnitude metrics: comparison cheat sheet

Metric	Units	Outlier penalty	Best for	Watch out for
MSE	y^2	quadratic	training loss	not interpretable
RMSE	y	quadratic	reporting in real units	inflated by outliers
MAE	y	linear	robust point estimate	ignores error direction
MedAE	y	zero	"typical" error	ignores half the data
RMSLE	log-units	relative	skewed positive targets	undefined for $y < 0$

Lower is better for all five. None of them can be compared across datasets with different y scales (Yerevan rent in kAMD vs. Berlin rent in EUR).

Two-line rule of thumb: report **RMSE** if outliers matter to you, **MAE** if they don't, **RMSLE** if the target is multiplicative.

R^2 and adjusted R^2

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

- ▶ R^2 : fraction of variance explained, relative to the constant-mean baseline. $R^2 = 1$ is perfect; $R^2 = 0$ means “no better than predicting $\bar{y}$ ”.
- ▶ **Adjusted R^2** penalizes adding features that don't help. R^2 *always* increases when you add a feature; adjusted R^2 can decrease.
- ▶ Scale-free $\Rightarrow$ comparable across datasets (unlike MAE/RMSE).

Generalized R^2 . The same skeleton works with any loss and any baseline:

$$R^2_{\text{gen}} = 1 - \frac{\text{Loss}_{\text{complex}}}{\text{Loss}_{\text{baseline}}}$$

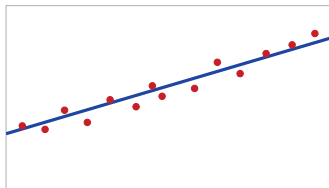
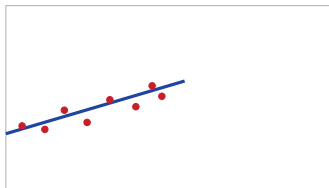
Classical R^2 is the special case (loss = SSE, baseline = constant mean). Use any loss, any reference. On test data, when the complex model is worse than the baseline, this is negative.

R^2 on test can be negative. It just means your model is worse than predicting the training mean. Investigate.

Better $R^2 \neq$ better fit

Same model, same MSE, two different views of the data. R^2 depends on the *range* of y ; MSE depends on the *magnitude* of residuals.

half range: $R^2 = 0.74$, $\text{MSE} = 0.02$ full range: $R^2 = 0.83$, $\text{MSE} = 0.02$



Why? $R^2 = 1 - \text{SSE}/\text{SST}$. Extending the range inflates SST much faster than SSE $\Rightarrow$ ratio shrinks $\Rightarrow R^2$ goes up - even though the fit didn't actually improve.

Lesson: R^2 is invariant under linear rescaling of y in a way MSE isn't. Don't compare R^2 across datasets with different y -ranges. Pair it with RMSE/MAE.

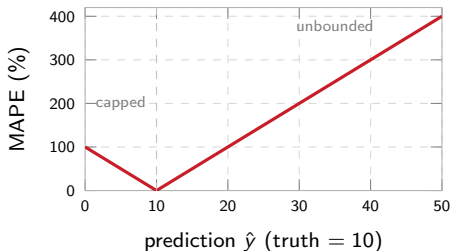
MAPE: the popular but flawed default

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$$

Popular because it reports as a percentage.
Stakeholders love it.

Three known problems:

1. **Undefined when** $y_i = 0$. Any zero target $\Rightarrow$ division by zero $\Rightarrow$ infinite MAPE.
2. **Asymmetric**. Under-prediction capped at 100%; over-prediction unbounded. Plot shows this for fixed truth $y = 10$.
3. **Biases models toward under-prediction** when used as a loss.



small y dominates



V: asymmetric around $y = 10$.

Bottom: small y blows up the percent.

SMAPE: the symmetric fix

$$\text{SMAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

- ▶ Symmetric: a 50% over-prediction and a 50% under-prediction get the same score.
- ▶ Bounded in $[0\%, 200\%]$ - no infinities.
- ▶ Defined when $y_i = 0$ (as long as $\hat{y}_i \neq 0$).
- ▶ Still has its own quirks (the bound of 200% means a missed prediction of 0 dominates), but generally better-behaved than MAPE.

```
import numpy as np
def smape(y, y_pred):
    return 200 * np.mean(np.abs(y - y_pred) / (np.abs(y) + np.abs(y_pred)))
```

MASE (Mean Absolute Scaled Error): MAE of the model divided by MAE of a naïve forecast (e.g., “predict last value”). Scale-free, robust to zeros, standard for time-series competitions (M4, M5).

MBE: catching systematic bias

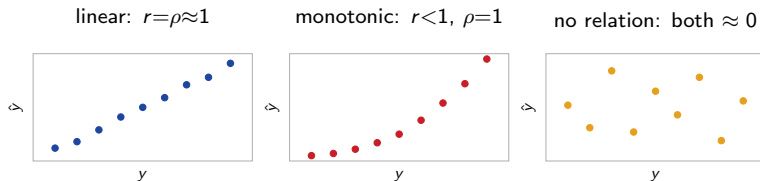
$$\text{MBE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

- ▶ **Positive** $\Rightarrow$ model systematically over-predicts.
- ▶ **Negative** $\Rightarrow$ model systematically under-predicts.
- ▶ Symmetric error metrics (MSE, MAE) *hide* bias - +50 and -50 errors cancel in MBE but add in MAE.
- ▶ Critical for forecasting and demand planning, where a 1% persistent bias compounds across millions of items.

Rule: report MBE alongside MAE / RMSE whenever the cost of over-prediction differs from the cost of under-prediction.

Yerevan example: a rent-estimator with $\text{MBE} = +25$ kAMD over-prices every listing. Landlords love it; the marketplace loses to competitors with honest estimators.

Pearson, Spearman, Kendall: correlations



Pearson r - linear co-movement: $r = \text{cov}(y, \hat{y}) / (\sigma_y \sigma_{\hat{y}})$.
High when relationship is straighty.

Spearman ρ - Pearson on *ranks*. Robust to monotonic transforms; doesn't need linearity.

Kendall τ - fraction of concordant pairs minus discordant. Robust to ties, slower to compute.

```
from scipy.stats import (  
    pearsonr, spearmanr, kendalltau  
)  
r, _ = pearsonr(y, y_pred)  
rho, _ = spearmanr(y, y_pred)  
tau, _ = kendalltau(y, y_pred)
```

Predict-first: when correlation lies

Your model's predictions on a 3-row test set:

y_{true}	100	200	300
$\hat{y}$	1100	1200	1300

Predict-first: when correlation lies

Your model's predictions on a 3-row test set:

y_{true}	100	200	300
$\hat{y}$	1100	1200	1300

Predict:

- ▶ Pearson correlation r between y and $\hat{y}$?
- ▶ MAE?
- ▶ Would you deploy this model?

Predict-first: when correlation lies

Your model's predictions on a 3-row test set:

y_{true}	100	200	300
$\hat{y}$	1100	1200	1300

Predict:

- ▶ Pearson correlation r between y and $\hat{y}$?
- ▶ MAE?
- ▶ Would you deploy this model?

Answers:

- ▶ **Pearson** $r = 1.0$. The predictions are perfectly linearly related to truth - they're just shifted by +1000.
- ▶ **MAE** = 1000. The predictions are uniformly terrible.
- ▶ No.

Correlation measures association, not accuracy. Use correlations for screening features. Use MAE/RMSE for evaluating models.

AIC and BIC: model comparison, not evaluation

$$\text{AIC} = 2k - 2\ln(\hat{L}) \quad \text{BIC} = k\ln(n) - 2\ln(\hat{L})$$

where k = number of model parameters, n = sample size, $\hat{L}$ = maximized likelihood.

- ▶ Both compare *candidate models on the same data*. Lower is better.
- ▶ They balance **fit quality** ($-2\ln \hat{L}$) against **model complexity** (k).
- ▶ Require a probabilistic model so $\hat{L}$ is defined - typical with statsmodels OLS, not directly with sklearn estimators.
- ▶ These are *model selection* criteria, not accuracy metrics. They tell you which model to pick; they do not tell you how accurate that model is on unseen data.

```
import statsmodels.api as sm
res = sm.OLS(y, sm.add_constant(X)).fit()
print(res.aic, res.bic)
```

AIC vs. BIC: when each wins

Both penalize complexity, but **BIC penalizes harder** once $n \geq 8$ (because $\ln n > 2$).

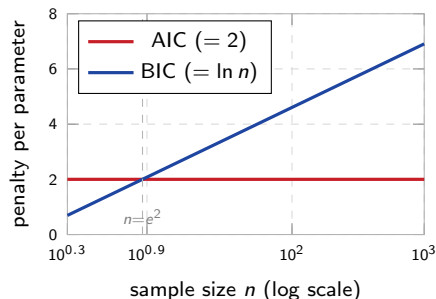
AIC ($2k$ per param):

- ▶ Better for *prediction* - keeps useful features.
- ▶ Asymptotically equivalent to leave-one-out CV.

BIC ($\ln(n) \cdot k$ per param):

- ▶ Better for *model identification* - conservative.
- ▶ Consistent: as $n \rightarrow \infty$, BIC picks the true model.

Practical: report both. When they disagree, BIC is more conservative. Pair with CV for honest accuracy.



They cross at $n = e^2 \approx 7.4$. Beyond that, BIC's complexity tax is harsher.

The four standard diagnostic plots

Numerical metrics give you *one number*. Plots show you *what kind of wrong* your model is. Four standard plots (the R `plot.lm` set):

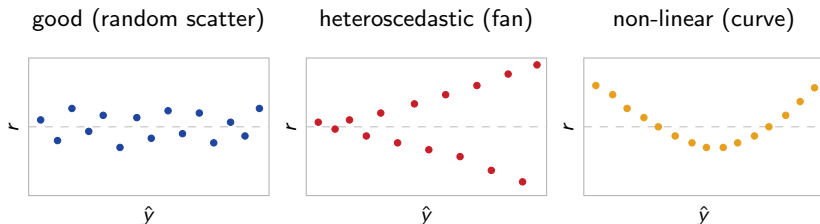
- ▶ **Residuals vs. fitted.** Are residuals patterned? (linearity, heteroscedasticity)
- ▶ **Scale-Location.** Does residual spread depend on prediction? (heteroscedasticity, refined)
- ▶ **Normal QQ.** Are residuals roughly Gaussian? (matters for OLS inference)
- ▶ **Residuals vs. leverage.** Which observations have unusual influence?

Look at all four. A model with perfect $R^2 = 0.97$ can still be broken in ways the diagnostic plots reveal.

`statsmodels` produces these via `sm.graphics.plot_regress_exog` or by manual `matplotlib`.

Residuals vs. fitted

Residual $r_i = y_i - \hat{y}_i$ plotted against the prediction $\hat{y}_i$. Should look like random scatter around 0.

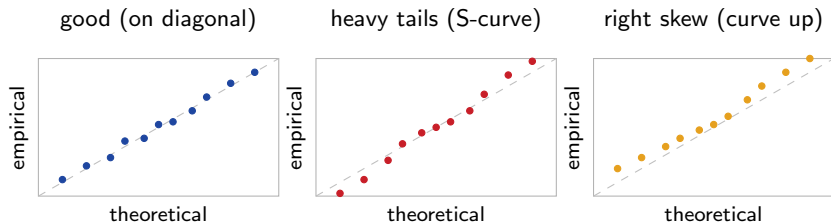


Fan shape: variance grows with prediction (heteroscedasticity). Fix: log-transform y , use weighted least squares, or use robust standard errors.

Curve: you're missing a non-linear term. Fix: add polynomial features, interaction terms, or switch to a tree.

Normal QQ plot

Plots empirical residual quantiles against theoretical Normal quantiles. Should fall on the diagonal if residuals are Gaussian.



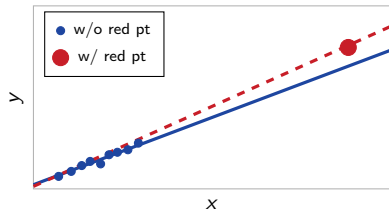
Why care? OLS *point estimates* don't need normality. OLS *inference* (CIs, t-tests, F-tests) does. Heavy tails or skew $\Rightarrow$ p-values are wrong, but predictions are fine.

Leverage and Cook's distance

Leverage h_i . How unusual is $\mathbf{x}^{(i)}$? Diagonal of the hat matrix $\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. High leverage $\Rightarrow$ this point can pull the fit strongly toward itself.

Cook's distance D_i . How much would $\hat{\theta}$ change if you removed observation i ? Combines leverage AND residual size.

$$D_i = \frac{r_i^2}{p \cdot \text{MSE}} \cdot \frac{h_i}{(1 - h_i)^2}$$



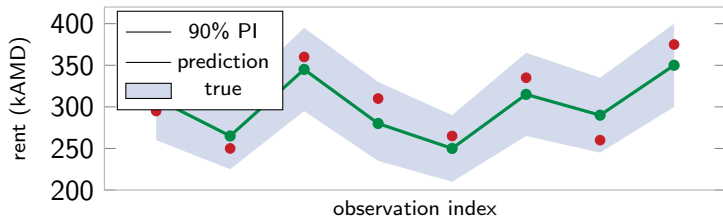
The high-leverage red point pulls the line toward itself.

Rule of thumb: $D_i > 4/n$ is suspicious. Investigate before deleting. High-Cook's points are often (a) data-entry errors, (b) genuinely unusual but real cases (Yerevan 200 m² penthouse in training), or (c) telling you the model class is too rigid.

Point predictions aren't enough

“This apartment will rent for 310 kAMD” is a *point prediction*. It says nothing about uncertainty.

“90% prediction interval: [265, 355] kAMD” is **honest** - it communicates what the model knows and doesn't know.



Out of 8 truths, all 8 fell within the 90% PI - actual coverage 100%. With a larger test set we'd expect roughly 90%.

Coverage: testing a prediction interval

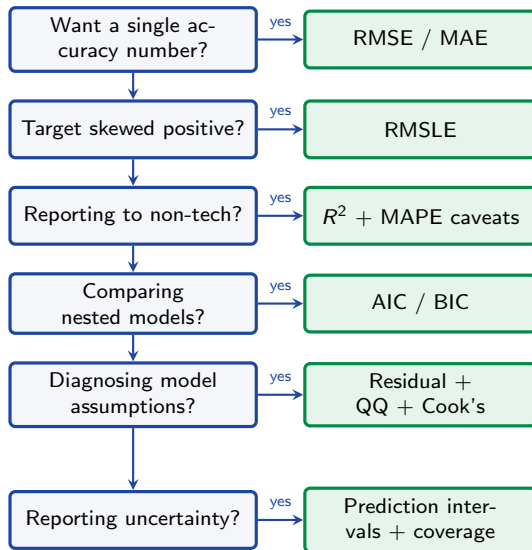
Empirical coverage = fraction of true values that fell inside their predicted interval.

For a well-calibrated $1 - \alpha$ PI, coverage on a held-out test set should be approximately $1 - \alpha$. If you build 90% PIs and only 75% of test points land inside, your intervals are too narrow - you're claiming more confidence than you have.

```
# Quick way: residual quantiles on a calibration split
residuals = y_cal - model.predict(X_cal)
q_lo, q_hi = np.quantile(residuals, [0.05, 0.95])      # 90% PI
y_pred = model.predict(X_test)
lo, hi = y_pred + q_lo, y_pred + q_hi
coverage = np.mean((y_test >= lo) & (y_test <= hi))
print(f"empirical coverage = {coverage:.2%}")          # should be ~90%
```

Modern tool: MAPIE library implements *conformal prediction*, which gives provable coverage guarantees regardless of model.

Picking the right metric



Recap

- ▶ **Magnitude metrics** (MSE, RMSE, MAE, MedAE, RMSLE) all answer “how big are the errors?” but differ in how they treat outliers and scale. Report at least two together.
- ▶ **Relative metrics** (R^2 , adjusted R^2 , MAPE, SMAPE, MBE) are scale-free or normalized. R^2 can be negative on test data; MAPE is asymmetric and ill-defined at $y = 0$; MBE catches bias the symmetric metrics hide.
- ▶ **Correlations** measure association, not accuracy. A perfect Pearson can coexist with terrible predictions.
- ▶ **AIC / BIC** are for model *comparison*, not absolute accuracy. Pair them with CV.
- ▶ **Diagnostic plots** reveal what kind of wrong your model is. Residuals vs. fitted, QQ, scale-location, leverage-vs-Cook's.
- ▶ **Prediction intervals + coverage** turn point predictions into honest uncertainty reports.

One model. Many metrics. Report the ones that match what your stakeholder will actually do with the output.

HW01f - practice

Estimated time: 90-120 minutes. Tasks 1-4 graded; bonuses optional.

1. On the Yerevan rent dataset: fit Ridge with reasonable defaults. Compute MSE, RMSE, MAE, MedAE, R^2 , MAPE on the test set. Report all six and discuss which one you'd lead with for (a) marketplace stakeholders, (b) ML team review.
2. Inject 5 outliers into the training data. Refit. Recompute the six metrics. Which moved the most? Which moved the least? Why?
3. Plot the four standard diagnostic plots for your fitted model. Identify any high-Cook's-distance points.
4. Build a 90% prediction interval using the residual-quantile method. Report empirical coverage on test.
5. *Bonus 1*: log-transform rent and re-fit. Report RMSLE before and after. Compare to RMSE on the original scale.
6. *Bonus 2*: install MAPIE and produce conformal 90% intervals. Compare coverage to your residual-quantile intervals.
7. *Bonus 3*: fit two candidate models in `statsmodels`: one with 5 features, one with 10. Compare AIC, BIC, and 5-fold CV MSE. Do they agree? **Most instructive question.**